

Résumé Complet - Développement Android

Interface Utilisateur, Persistance, Communication, API

AmineGR03

29 janvier 2026

Table des matières

Chapitre 3 : Interface Utilisateur sous Android

Introduction aux Vues et Contrôles

Concepts fondamentaux

Vue (View) : Chaque élément visible et interactif sur l'écran d'une application Android est une instance de la classe **View** ou d'une de ses sous-classes. Que ce soit un bouton, un champ de texte ou une image, tout est une "Vue".

ViewGroup : Permet de regrouper des éléments graphiques. Des ViewGroup particuliers sont prédéfinis : ce sont des gabarits (layout) qui proposent une prédisposition des objets graphiques.

Déclaration XML : Les déclarations se font principalement en XML, ce qui évite de passer par les instantiations Java.

Contrôles basiques

- **TextView** : Le contrôle le plus simple pour afficher du texte statique. Il est non éditable par l'utilisateur.
 - **android:text** : Le contenu textuel
 - **android:textSize** : La taille du texte
- **EditText** : Permet aux utilisateurs de saisir du texte, essentiel pour les formulaires et les champs de recherche.
 - **android:hint** : Texte d'indication affiché si le champ est vide
 - **android:inputType** : Spécifie le type de données attendu (ex : texte, nombre)
- **Button** : Contrôle interactif qui déclenche une action lorsqu'il est pressé.
 - **android:text** : Le texte affiché sur le bouton
 - **android:onClick** : Permet de lier l'action à une méthode dans le code

Contrôles de sélection

- **CheckBox** : Choix multiples. Permet à l'utilisateur de sélectionner ou de désélectionner une ou plusieurs options indépendamment.
- **RadioGroup + RadioButton** : Choix unique. Le RadioGroup regroupe des RadioButton pour assurer un choix exclusif parmi un ensemble d'options. Les RadioButton doivent être contenus dans un RadioGroup.

Positionnement et Layouts

Types de Layouts

- **LinearLayout** : Organisation simple des vues en ligne (horizontale ou verticale). Idéal pour les listes et les arrangements linéaires simples.
- **ConstraintLayout** : Positionnement flexible basé sur des contraintes relatives. Recommandé pour toutes les interfaces complexes et performantes.
- **FrameLayout** : Utilisé pour afficher un seul élément ou superposer plusieurs vues. Idéal pour les fragments ou les superpositions d'éléments.
- **TableLayout** : Organisation des vues en lignes et colonnes, similaire à une table HTML. Convient pour l'affichage de données tabulaires.
- **RelativeLayout** : Positionnement relatif. Permet de positionner chaque élément enfant par rapport aux bords de son conteneur parent ou par rapport à d'autres vues.

Attributs des Layouts

- `android:layout_width` et `android:layout_height` :
 - `"match_parent"` : L'élément remplit tout l'espace disponible de son parent
 - `"wrap_content"` : L'élément prend la taille minimale nécessaire pour contenir son contenu
- `android:orientation` : Spécifie la direction d'empilement des vues enfants pour les layouts linéaires (`LinearLayout`).
 - `"vertical"` : Les éléments sont empilés de haut en bas
 - `"horizontal"` : Les éléments sont placés côte à côte de gauche à droite
- `android:gravity` : Contrôle l'alignement du contenu ou des vues enfants au sein du Layout.

Exemple LinearLayout

```

1 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
2   android:orientation="vertical"
3   android:layout_width="match_parent"
4   android:layout_height="match_parent"
5   android:gravity="center"
6   android:id="@+id/accueilid">
7   <!-- Vos contr les ici -->
8 </LinearLayout>

```

Identificateurs (ID) Android

- `android:id="@+id/idElement"` : Utilisation lors de la déclaration d'un nouvel élément dans le XML
- `@id/idElement` : Référence à un élément existant pour le positionnement ou la manipulation

Élasticité et Adaptabilité des UI

Unités de mesure

- **dp (density-independent pixel)** : Le dp est une unité virtuelle qui s'adapte automatiquement à la densité de l'écran. Il est l'unité préférée pour spécifier les tailles des éléments, les marges et le padding afin d'assurer une apparence uniforme sur différents appareils.
- **sp (scale-independent pixel)** : Similaire au dp, le sp s'ajuste non seulement à la densité de l'écran mais aussi aux préférences de taille de texte de l'utilisateur (paramètres d'accessibilité). Il est donc essentiel pour les tailles de texte.
- **px (pixel)** : Le px représente un pixel réel de l'écran. Sa taille physique varie énormément selon la densité de l'appareil, rendant les mises en page incohérentes. Il est fortement déconseillé pour la conception d'interfaces utilisateur adaptatives.

Bonnes pratiques UI adaptatives

- Toujours utiliser **dp** et **sp**
- Éviter les tailles fixes en pixels (px)
- Utiliser `ConstraintLayout` pour s'adapter aux écrans
- Créer des ressources alternatives (`/layout-sw600dp/` pour les tablettes)
- Tester avec l'émulateur sur différentes résolutions

ConstraintLayout

Le **ConstraintLayout** est un gabarit de mise en page moderne et flexible qui permet de créer des interfaces utilisateur responsives et fluides. Il utilise un système de contraintes pour positionner et dimensionner les vues.

Types de Contraintes

- **Contraintes Relatives** : Positionnent une vue par rapport aux bords ou aux lignes de base d'une autre vue
 - `layout_constraintStart_toStartOf`
 - `layout_constraintTop_toTopOf`
 - `layout_constraintBaseline_toBaselineOf`
- **Contraintes Avancées** :
 - **Contraintes Circulaires** : `layout_constraintCircle`, `layout_constraintCircleRadius`, `layout_constraintCircleAngle`
 - **Chaînes (Chains)** : `layout_constraintHorizontal_chainStyle` (`spread`, `spread_inside`, `packed`)
 - **Barrières (Barriers)** : Créent une ligne de référence virtuelle basée sur plusieurs vues
 - **Guidelines** : Lignes de référence invisibles pour l'alignement

Listes et Adaptateurs

Définition

Les **Adaptateurs** font le lien entre une source de données et une vue (liste, grille, etc.), permettant d'afficher des données complexes de manière efficace dans des composants d'interface utilisateur.

Principe du pattern Adapter

Permet à des interfaces incompatibles de collaborer. Il s'interpose entre les données brutes et le composant UI, adaptant les données pour l'affichage.

Types courants

- **ArrayAdapter** : Pour les données simples comme des chaînes de caractères
- **BaseAdapter** : Pour les vues personnalisées
- **RecyclerView.Adapter** : Pour des listes plus performantes et flexibles

Exemple avec ListView

```
1 ListView listView = findViewById(R.id.listView);
2 String[] villes = {"Rabat", "Agadir", "Safi"};
3 ArrayAdapter<String> adapter = new ArrayAdapter<>(
4     this,
5     android.R.layout.simple_list_item_1,
6     villes
7 );
8 listView.setAdapter(adapter);
```

RecyclerView

Le **RecyclerView** est la version moderne et plus performante que **ListView** pour l’affichage de grandes collections de données. Il offre une meilleure gestion de la mémoire et une fluidité accrue grâce à son architecture modulaire.

Composants du RecyclerView :

- **Adapter** : Définit la structure des éléments et assure la liaison entre les données et les vues
- **ViewHolder** : Réutilise les vues existantes pour un élément de liste, optimisant ainsi les performances de défilement
- **LayoutManager** : Définit la manière dont les éléments sont disposés (liste linéaire, grille ou cascade)

Exemple RecyclerView

```
1 // Dans MainActivity.java
2 RecyclerView recyclerView = findViewById(R.id.my_recycler_view);
3 recyclerView.setLayoutManager(new LinearLayoutManager(this));
4 recyclerView.setAdapter(new MonAdapter(vosDonnees));
```

Adapter personnalisé

```
1 public class MonAdapter extends RecyclerView.Adapter<MonAdapter.
   MonViewHolder> {
2     private String[] donnees;
3
4     public MonAdapter(String[] donnees) {
5         this.donnees = donnees;
6     }
7
8     @Override
9     public MonViewHolder onCreateViewHolder(@NonNull ViewGroup parent,
10        int viewType) {
11         View view = LayoutInflater.from(parent.getContext())
12             .inflate(R.layout.item_layout, parent, false);
13         return new MonViewHolder(view);
14     }
15
16     @Override
17     public void onBindViewHolder(@NonNull MonViewHolder holder, int
18        position) {
19         holder.textView.setText(donnees[position]);
20     }
21
22     @Override
23     public int getItemCount() {
24         return donnees.length;
25     }
26
27     public static class MonViewHolder extends RecyclerView.ViewHolder {
28         public TextView textView;
29
30         public MonViewHolder(@NonNull View itemView) {
31             super(itemView);
32             textView = itemView.findViewById(R.id.item_text_view);
33         }
34     }
35 }
```

```

31     }
32 }
33 }

```

Navigation entre Activités

Intent

Un **Intent** est un message abstrait qui permet de démarrer une autre activité, un service, ou de diffuser un événement. Il agit comme un messenger entre les composants de l'application ou même entre différentes applications. Il peut transporter des données entre les activités.

Navigation simple

```

1 Intent i = new Intent(this, SecondActivity.class);
2 startActivity(i);

```

Transfert de données

```

1 // Envoi
2 Intent i = new Intent(this, SecondActivity.class);
3 i.putExtra("nom", "Ali");
4 startActivity(i);
5
6 // Récupération
7 String nom = getIntent().getStringExtra("nom");

```

Gestion des Événements

OnClickListener

Le gestionnaire d'événements le plus fondamental en Android est l'**OnClickListener**, utilisé pour détecter les clics des utilisateurs sur divers composants d'interface utilisateur.

```

1 Button monBouton = findViewById(R.id.mon_bouton);
2 monBouton.setOnClickListener(new View.OnClickListener() {
3     @Override
4     public void onClick(View v) {
5         Toast.makeText(MonActivite.this, "Bouton cliqué !",
6             Toast.LENGTH_SHORT).show();
7     }
8 });

```

OnFocusChangeListener

Les événements de focus sont essentiels pour gérer l'interaction des utilisateurs avec des champs de saisie de texte.

```

1 EditText monChampTexte = findViewById(R.id.mon_champ_texte);
2 monChampTexte.setOnFocusChangeListener(new View.OnFocusChangeListener()
3 {
4     @Override
5     public void onFocusChange(View v, boolean hasFocus) {
6         if (hasFocus) {

```

```

6         // Le composant a gagn le focus
7     } else {
8         // Le composant a perdu le focus
9         // Validation possible ici
10    }
11 }
12 });

```

TextWatcher

Pour détecter et réagir aux changements dans un champ de saisie de texte (EditText).

```

1 EditText monEditText = findViewById(R.id.mon_edit_text);
2 monEditText.addTextChangedListener(new TextWatcher() {
3     @Override
4     public void beforeTextChanged(CharSequence s, int start, int count,
5         int after) { }
6
7     @Override
8     public void onTextChanged(CharSequence s, int start, int before, int
9         count) {
10         Log.d("TextWatcher", "Pendant modification: " + s.toString());
11     }
12
13     @Override
14     public void afterTextChanged(Editable s) { }
15 });

```

Les Menus

Menus d'options

Les menus d'options sont définis dans des fichiers XML stockés dans le dossier `/res/menu/`.

```

1 <!-- res/menu/main_menu.xml -->
2 <menu>
3     <item android:id="@+id/action_settings"
4         android:title="Param tres" />
5 </menu>

```

```

1 @Override
2 public boolean onCreateOptionsMenu(Menu menu) {
3     getMenuInflater().inflate(R.menu.main_menu, menu);
4     return true;
5 }

```

Menus contextuels

Les menus contextuels apparaissent lorsque l'utilisateur effectue un clic long sur un élément de l'interface.

```

1 TextView myTextView = findViewById(R.id.my_text_view);
2 registerForContextMenu(myTextView);
3
4 @Override
5 public void onCreateContextMenu(ContextMenu menu, View v,
6     ContextMenu.ContextMenuInfo menuInfo) {

```

```

7      super.onCreateContextMenu(menu, v, menuInfo);
8      if (v.getId() == R.id.my_text_view) {
9          getMenuInflater().inflate(R.menu.context_menu, menu);
10     }
11 }
12
13 @Override
14 public boolean onOptionsItemSelected(MenuItem item) {
15     switch (item.getItemId()) {
16         case R.id.action_edit:
17             Toast.makeText(this, "Action Modifier",
18                 Toast.LENGTH_SHORT).show();
19             return true;
20         case R.id.action_delete:
21             Toast.makeText(this, "Action Supprimer",
22                 Toast.LENGTH_SHORT).show();
23             return true;
24         default:
25             return super.onOptionsItemSelected(item);
26     }
27 }

```

Dialogue avec l'Utilisateur

Toast

Le Toast est le moyen le plus simple d'afficher un message bref à l'utilisateur.

```

1 Toast.makeText(this, "This is a Toast Message",
2     Toast.LENGTH_SHORT).show();

```

AlertDialog

L'AlertDialog est un composant puissant pour créer des boîtes de dialogue modales.

```

1 new AlertDialog.Builder(this)
2     .setTitle("Confirmation")
3     .setMessage("Voulez-vous quitter ?")
4     .setPositiveButton("Oui", (d, i) -> finish())
5     .setNegativeButton("Non", null)
6     .show();

```

Snackbar

Les Snackbars sont la solution moderne recommandée par Google pour afficher des messages temporaires.

```

1 Snackbar.make(findViewById(R.id.main), "Donn es enregistr es",
2     Snackbar.LENGTH_SHORT).show();

```

Chapitre 4 : Persistance des Données

Introduction

Comprendre comment une application Android peut conserver ses données au-delà d'une seule exécution, garantissant la disponibilité et l'intégrité des informations même après la fermeture de l'application ou le redémarrage de l'appareil.

Accès aux Fichiers

Types de stockage

- **Stockage Interne** : Cet espace est propre à l'application, ce qui signifie que les données y sont isolées et non accessibles par d'autres applications. Il est fortement recommandé pour le stockage des informations confidentielles ou des données essentielles au fonctionnement exclusif de votre application.
- **Stockage Externe** : Ce stockage est partagé avec d'autres applications et est donc adapté aux fichiers volumineux ou aux contenus que l'utilisateur souhaite partager (photos, documents). L'accès à cet espace nécessite la gestion appropriée des permissions utilisateur.

Bonnes pratiques pour la gestion des fichiers

1. **Vérification d'existence** : Toujours vérifier la présence d'un fichier avant toute tentative de lecture
2. **Fermeture des flux** : Il est crucial de fermer systématiquement tous les flux (`InputStream`, `OutputStream`) après usage
3. **Protection des données sensibles** : Pour les informations confidentielles, utilisez le stockage interne
4. **Gestion des erreurs** : Mettez en place des blocs `try-catch` robustes
5. **Maintenance des fichiers** : Prévoyez des mécanismes pour la suppression ou la mise à jour régulière des fichiers obsolètes

SharedPreferences

Les `SharedPreferences` sont un mécanisme simple pour stocker des données légères sous forme de paires clé-valeur, utilisées principalement pour les préférences utilisateur et les réglages d'application.

Obtenir l'Instance

```
1 SharedPreferences prefs = getSharedPreferences("MonAppPrefs",
2 Context.MODE_PRIVATE);
```

Lire des Données

```
1 String nom = prefs.getString("nomUtilisateur", "Invit ");
2 int age = prefs.getInt("age", 0);
3 boolean notifs = prefs.getBoolean("notificationsActives", false);
```

Écrire des Données

```

1 SharedPreferences.Editor editor = prefs.edit();
2 editor.putString("nomUtilisateur", "John Doe");
3 editor.putInt("age", 30);
4 editor.putBoolean("notificationsActives", true);
5 editor.apply(); // ou editor.commit()

```

Supprimer des Données

```

1 editor.remove("nomUtilisateur");
2 editor.apply();

```

apply() vs commit()

- **apply()** : Asynchrone, plus rapide, pas de valeur de retour
- **commit()** : Synchrone, bloque le thread, retourne un booléen

Recommandation : Utiliser **apply()** dans la plupart des cas.

Bases de Données SQLite

SQLite est une solution de base de données relationnelle légère et efficace, idéale pour la persistance des données directement sur l'appareil Android.

Caractéristiques

- **Base de données Embarquée** : SQLite est entièrement intégrée à l'application
- **Stockage Structuré** : Permet d'organiser et de stocker des données dans des tables
- **Autonome et Locale** : Absence de serveur, fonctionnement hors ligne complet
- **Compatible SQL** : Supporte la majorité des commandes SQL standards

SQLiteOpenHelper

SQLiteOpenHelper est une classe utilitaire fournie par Android pour faciliter la création, l'ouverture et la gestion des versions de votre base de données.

Création d'une classe DatabaseHelper

```

1 public class DatabaseHelper extends SQLiteOpenHelper {
2     public static final String DATABASE_NAME = "MaBaseDeDonnees.db";
3     public static final int DATABASE_VERSION = 1;
4
5     public DatabaseHelper(Context context) {
6         super(context, DATABASE_NAME, null, DATABASE_VERSION);
7     }
8
9     @Override
10    public void onCreate(SQLiteDatabase db) {
11        String CREATE_TABLE = "CREATE TABLE produits (" +
12                                "id INTEGER PRIMARY KEY AUTOINCREMENT," +
13                                "nom TEXT," +
14                                "prix REAL," +

```

```

15         "quantite INTEGER)";
16         db.execSQL(CREATE_TABLE);
17     }
18
19     @Override
20     public void onUpgrade(SQLiteDatabase db, int oldVersion, int
        newVersion) {
21         db.execSQL("DROP TABLE IF EXISTS produits");
22         onCreate(db);
23     }
24 }

```

Opérations CRUD

Insertion de Données

```

1 public boolean insertData(String nom, double prix, int quantite) {
2     SQLiteDatabase db = this.getWritableDatabase();
3     ContentValues contentValues = new ContentValues();
4     contentValues.put("nom", nom);
5     contentValues.put("prix", prix);
6     contentValues.put("quantite", quantite);
7     long result = db.insert("produits", null, contentValues);
8     return result != -1;
9 }

```

Lecture de Données

```

1 public Cursor getData() {
2     SQLiteDatabase db = this.getWritableDatabase();
3     Cursor res = db.rawQuery("SELECT * FROM produits", null);
4     return res;
5 }

```

Mise à Jour de Données

```

1 public boolean updateData(String id, String nom, double prix, int
    quantite) {
2     SQLiteDatabase db = this.getWritableDatabase();
3     ContentValues contentValues = new ContentValues();
4     contentValues.put("nom", nom);
5     contentValues.put("prix", prix);
6     contentValues.put("quantite", quantite);
7     db.update("produits", contentValues, "id = ?", new String[]{id});
8     return true;
9 }

```

Suppression de Données

```

1 public Integer deleteData(String id) {
2     SQLiteDatabase db = this.getWritableDatabase();
3     return db.delete("produits", "id = ?", new String[]{id});
4 }

```

Utilisation dans une Activity

```

1 public class MainActivity extends AppCompatActivity {
2     DatabaseHelper myDb;
3 }

```

```
4      @Override
5      protected void onCreate(Bundle savedInstanceState) {
6          super.onCreate(savedInstanceState);
7          setContentView(R.layout.activity_main);
8          myDb = new DatabaseHelper(this);
9
10         // Exemple d'insertion
11         boolean isInserted = myDb.insertData("Livre", 19.99, 50);
12         if (isInserted) {
13             Toast.makeText(this, "Donn es ins r es",
14                 Toast.LENGTH_LONG).show();
15         }
16     }
17 }
```

Chapitre 5 : Communication et Threads

Thread Principal (UI Thread)

Le thread principal (UI Thread) est responsable de l’affichage de l’interface utilisateur et de la gestion des interactions. Il ne doit jamais être bloqué par des opérations longues.

Multi-threading

Le multi-threading permet d’exécuter des tâches en arrière-plan sans bloquer l’interface utilisateur, garantissant une expérience utilisateur fluide.

Handler

Le **Handler** permet de communiquer entre threads et d’exécuter du code sur le thread principal depuis un thread secondaire.

AsyncTask (Déprécié)

AsyncTask était une classe qui permettait d’exécuter des opérations en arrière-plan et de publier les résultats sur le thread UI. Elle est maintenant dépréciée depuis Android 11.

Recommandation moderne : Utiliser **Coroutines** (Kotlin) ou **ExecutorService** avec **Handler** (Java).

Chapitre 6 : Intents et Communication entre Composants

Introduction aux Intents

Un **Intent** est un message envoyé à Android pour déclencher une action. Il permet la communication entre :

- Les activités d’une même application
- Les services
- D’autres applications installées
- Des composants du système Android (ex. **AlarmManager**)

Les Intents Explicites

Permettent de lancer une activité précise (classe connue).

```
1 Intent intent = new Intent(this, DetailActivity.class);
2 startActivity(intent);
```

Utilisés pour la navigation interne à l'application. Chaque activité cible doit être déclarée dans le `AndroidManifest.xml`.

Les Intents Implicites

Décrivent l'action à accomplir, sans nommer le composant. Android choisit automatiquement une application capable de répondre.

```
1 Intent intent = new Intent(Intent.ACTION_VIEW,
2     Uri.parse("https://emsi.ma"));
3 startActivity(intent);
```

Composantes d'un Intent

- **Action** : `intent.setAction(Intent.ACTION_VIEW)`
- **Data (URI)** : `intent.setData(Uri.parse("https://emsi.ma"))`
- **Type** : `intent.setType("text/plain")`
- **Extras** : `intent.putExtra("key", "value")`
- **Category** : `intent.addCategory(Intent.CATEGORY_BROWSABLE)`
- **Flags** : `intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK)`

Exemples d'Actions Prédéfinies

- `Intent.ACTION_VIEW` : Ouvrir un lien, une image ou un contact
- `Intent.ACTION_DIAL` : Ouvrir le composeur téléphonique
- `Intent.ACTION_SEND` : Partager du contenu (email par exemple)
- `Intent.ACTION_PICK` : Sélectionner une image ou un contact

Exemples Pratiques

Ouvrir une Page Web

```
1 Uri webpage = Uri.parse("https://www.google.com");
2 Intent webIntent = new Intent(Intent.ACTION_VIEW, webpage);
3 startActivity(webIntent);
```

Faire un Appel Téléphonique

```
1 Uri number = Uri.parse("tel:0645127859");
2 Intent call = new Intent(Intent.ACTION_DIAL, number);
3 startActivity(call);
```

Partager du Texte

```
1 Intent shareIntent = new Intent(Intent.ACTION_SEND);
2 shareIntent.setType("text/plain");
3 shareIntent.putExtra(Intent.EXTRA_SUBJECT, "Sujet du message");
4 shareIntent.putExtra(Intent.EXTRA_TEXT, "Contenu partager");
5 startActivity(Intent.createChooser(shareIntent, "Partager via"));
```

Passage de Données entre Activités

Envoi d'Extras

```
1 Intent intent = new Intent(this, ProfileActivity.class);
2 intent.putExtra("username", "Ali");
3 startActivity(intent);
```

Récupération

```
1 String user = getIntent().getStringExtra("username");
```

Transmission d'Objets Complexes (Parcelable)

Parcelable est la méthode recommandée pour passer des objets.

```
1 // Envoi
2 intent.putExtra("user", userObject);
3
4 // Récupération
5 User user = getIntent().getParcelableExtra("user");
```

Activity Result API

La nouvelle méthode standard et recommandée pour gérer les résultats d'activité.

```
1 ActivityResultLauncher<Intent> launcher =
2     registerForActivityResult(
3         new ActivityResultContracts.StartActivityForResult(),
4         result -> {
5             if (result.getResultCode() == RESULT_OK) {
6                 Intent data = result.getData();
7                 // Traitement du résultat
8             }
9         });
```

Broadcast Receivers

Les Broadcasts diffusent des messages à plusieurs composants.

Types

- **Système** : Batterie, réseau, écran, démarrage, etc.
- **Personnalisé** : Créé par l'application

Exemple

```
1 // mission
2 Intent intent = new Intent("com.emsi.MON_ACTION_PERSONNALISEE");
3 sendBroadcast(intent);
4
5 // Reception
6 registerReceiver(new MyReceiver(),
7     new IntentFilter(Intent.ACTION_POWER_CONNECTED));
```

PendingIntent

Un Intent différé exécuté plus tard, souvent par le système. Utilisé pour :

- Notifications
- Alarmes
- App Widgets

```
1 PendingIntent pendingIntent = PendingIntent.getActivity(
2     this, 0, intent, PendingIntent.FLAG_IMMUTABLE);
```

Chapitre 7 : API Android

Content Providers et Accès aux Contacts

Les **Content Providers** constituent le mécanisme fondamental d'Android pour partager des données entre applications.

Permissions nécessaires

```
1 <uses-permission android:name="android.permission.READ_CONTACTS" />
2 <uses-permission android:name="android.permission.WRITE_CONTACTS" />
```

Exemple : Lire les contacts

```
1 Cursor cursor = getContentResolver().query(
2     ContactsContract.Contacts.CONTENT_URI,
3     null, null, null, null
4 );
5
6 while (cursor.moveToNext()) {
7     String name = cursor.getString(
8         cursor.getColumnIndex(ContactsContract.Contacts.DISPLAY_NAME)
9     );
10    Log.d("CONTACT", name);
11 }
12 cursor.close(); // Important : lib rer les ressources
```

Fonctions de Téléphonie

Appels téléphoniques

- **ACTION_DIAL** : Ouvre le dialer sans permission - l'utilisateur contrôle l'appel
- **ACTION_CALL** : Lance l'appel directement - nécessite **CALL_PHONE** permission

```
1 Intent intent = new Intent(Intent.ACTION_DIAL);  
2 intent.setData(Uri.parse("tel:0612345678"));  
3 startActivity(intent);
```

Envoi de SMS

```
1 SmsManager sms = SmsManager.getDefault();  
2 sms.sendTextMessage(  
3     "0612345678",  
4     null,  
5     "Bonjour depuis Android",  
6     null, null  
7 );
```

Permission nécessaire : **SEND_SMS**

Géolocalisation avec Android

Sources de localisation

- **GPS** : Précision élevée (5-10m) mais consommation batterie importante
- **Réseau mobile** : Précision moyenne (100-500m), consommation modérée
- **Wi-Fi** : Bonne précision en zone urbaine (20-50m), faible consommation

APIs de localisation

- **LocationManager** : API historique d'Android, accès direct aux fournisseurs de localisation
- **FusedLocationProvider** : API moderne recommandée par Google. Fusionne automatiquement les sources pour optimiser précision et batterie

Permissions de localisation

- **ACCESS_COARSE_LOCATION** : Localisation approximative via réseau et Wi-Fi
- **ACCESS_FINE_LOCATION** : Localisation précise via GPS
- **Background Location** : Depuis Android 10, permission séparée pour localisation en arrière-plan

Caméra

Android permet d'accéder à l'appareil photo pour capturer des photos et vidéos. L'accès nécessite des permissions appropriées et peut se faire via :

- Intent implicite (déléguer à l'app caméra système)
- Camera2 API (contrôle avancé)
- CameraX (bibliothèque moderne recommandée)

Multimédia

Sonneries et Vibrations

```
1 // Vibration
2 Vibrator vibrator = (Vibrator) getSystemService(Context.VIBRATOR_SERVICE);
3 vibrator.vibrate(500); // 500 millisecondes
4
5 // Sonnerie
6 MediaPlayer mediaPlayer = MediaPlayer.create(this, R.raw.son);
7 mediaPlayer.start();
```

Chapitre 8 : Communication Réseau sous Android

Établissement de Connexions

Android permet d'établir des connexions à des serveurs distants via HTTP/HTTPS.

Protocoles de Communication

HTTP/HTTPS

Les applications Android peuvent communiquer avec des serveurs web via les protocoles HTTP et HTTPS.

Web Services

REST et JSON

Les services web REST utilisent JSON pour l'échange de données. Android fournit des bibliothèques comme Retrofit et OkHttp pour faciliter ces interactions.

Retrofit

Retrofit est une bibliothèque moderne et puissante pour effectuer des appels réseau en Android.

```
1 // Interface de service
2 public interface ApiService {
3     @GET("users")
4     Call<List<User>> getUsers();
5 }
6
7 // Utilisation
8 Retrofit retrofit = new Retrofit.Builder()
9     .baseUrl("https://api.example.com/")
10     .addConverterFactory(GsonConverterFactory.create())
11     .build();
12
13 ApiService service = retrofit.create(ApiService.class);
14 Call<List<User>> call = service.getUsers();
15 call.enqueue(new Callback<List<User>>() {
16     @Override
17     public void onResponse(Call<List<User>> call, Response<List<User>>
18         response) {
19         // Traitement de la r ponse
20     }
21 });
```

```
19     }
20
21     @Override
22     public void onFailure(Call<List<User>> call, Throwable t) {
23         // Gestion des erreurs
24     }
25 });
```

Communication Réseau en Multitâches

Les opérations réseau doivent toujours être effectuées sur un thread secondaire pour ne pas bloquer le thread UI. Utiliser :

- `ExecutorService` avec `Handler`
- `Coroutines` (Kotlin)
- `Retrofit` (gère automatiquement les threads)

Chapitre 9 : Déployer et Sécuriser des Applications Android

Préparation

Comprendre les distinctions entre les versions Debug et Release, la configuration adéquate, la gestion des logs et le versioning.

Signature

Maîtriser les Keystores, la gestion des clés et certificats, ainsi que les bonnes pratiques de sécurité.

Création d'un Keystore

```
1 keytool -genkey -v -keystore mon-keystore.jks
2     -keyalg RSA -keysize 2048 -validity 10000
3     -alias mon-alias
```

Publication

Utiliser efficacement la Play Console, préparer les App Bundles (AAB) et APK, et effectuer des tests internes rigoureux avant le lancement.

Déploiement

Gérer les différentes versions et canaux de déploiement, assurer un suivi précis des performances et planifier les mises à jour.

Processus complet

Le déploiement et la sécurisation d'applications Android sont des étapes fondamentales qui exigent une attention méticuleuse à chaque phase. D'une préparation rigoureuse de la version à la gestion stratégique de sa publication et de son déploiement via la Play Console, chaque détail compte. La signature sécurisée de l'application est un pilier central, garantissant l'intégrité et l'authenticité du code.

Concepts Clés à Retenir

Architecture Android

- **Activities** : Écrans de l'application
- **Fragments** : Composants réutilisables d'interface
- **Services** : Tâches en arrière-plan
- **Broadcast Receivers** : Réception d'événements système
- **Content Providers** : Partage de données

Cycle de Vie

- **onCreate()** : Initialisation de l'activité
- **onStart()** : L'activité devient visible
- **onResume()** : L'activité est au premier plan
- **onPause()** : L'activité perd le focus
- **onStop()** : L'activité n'est plus visible
- **onDestroy()** : Destruction de l'activité

Persistance des Données

- **SharedPreferences** : Données légères clé-valeur
- **Fichiers** : Stockage interne/externe
- **SQLite** : Base de données relationnelle
- **Room** : Bibliothèque moderne pour SQLite (recommandée)

Communication

- **Intents** : Communication entre composants
- **Broadcasts** : Messages système et personnalisés
- **HTTP/HTTPS** : Communication réseau
- **Web Services** : REST, JSON, SOAP

Sécurité

- **Permissions** : Runtime permissions depuis Android 6.0
- **Signature** : Keystore pour signer les applications
- **ProGuard** : Obfuscation du code
- **Chiffrement** : Protection des données sensibles

Conclusion

Le développement Android nécessite une compréhension approfondie de l'architecture du système, des composants de base, de la gestion des données et de la communication. En maîtrisant ces concepts fondamentaux, vous serez capable de créer des applications robustes, performantes et sécurisées qui offrent une excellente expérience utilisateur.

Les bonnes pratiques incluent :

- Utiliser des layouts adaptatifs (ConstraintLayout, dp/sp)

- Gérer correctement le cycle de vie des composants
- Effectuer les opérations longues sur des threads secondaires
- Demander les permissions au runtime
- Tester sur différents appareils et versions Android
- Optimiser les performances et la consommation de batterie